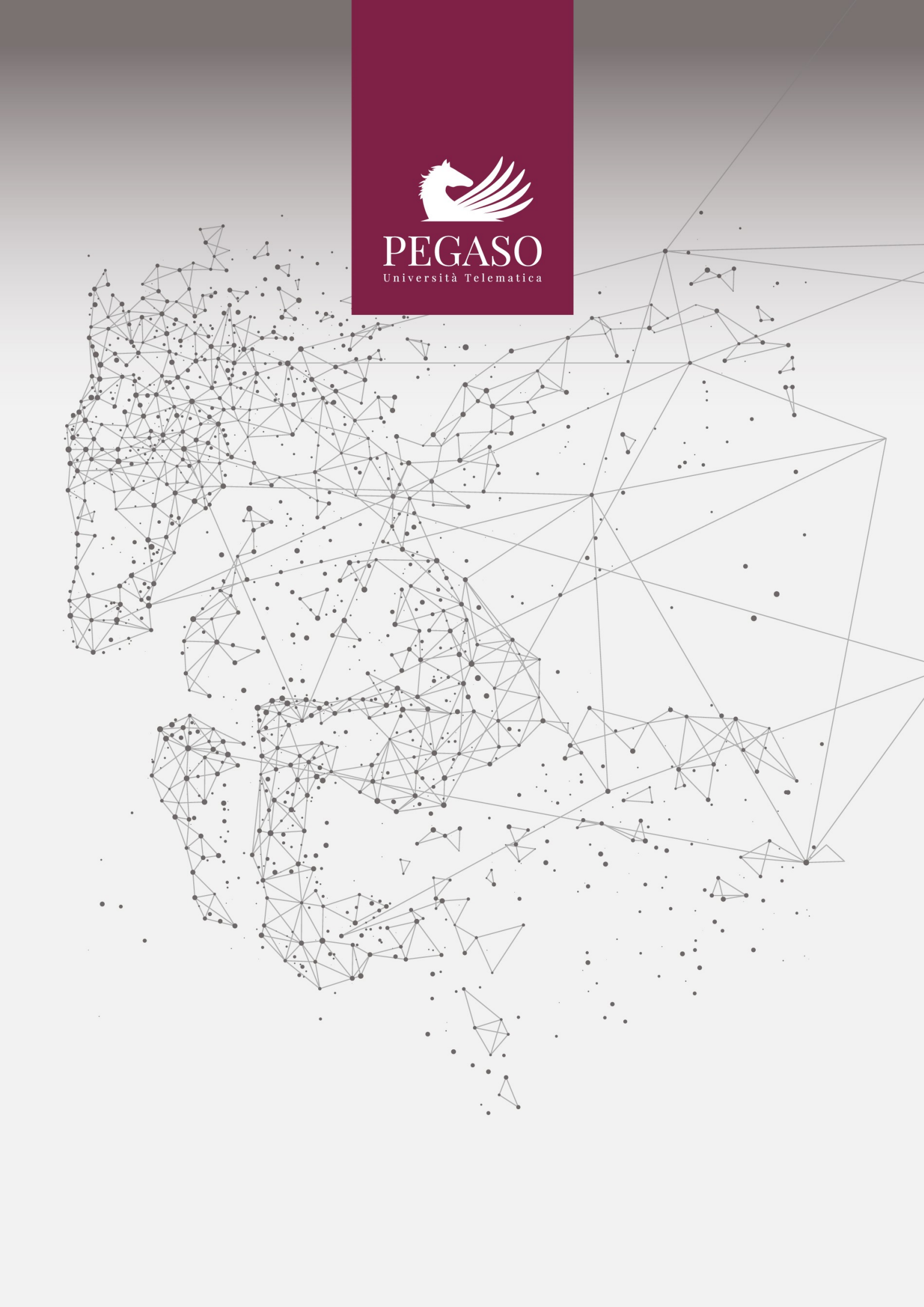

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. SISTEMA CLIENT SERVER 3
2. API (APPLICATION PROGRAM INTERFACE) 6
3. SOAP E REST 9
4. LA POPOLARITÀ DI REST 11
BIBLIOGRAFIA E SITOGRAFIA 13

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Sistema Client Server

Il termine sistema Client - Server indica un'architettura di rete nella quale genericamente un computer client si connette ad un server per la fruizione di un certo servizio, quale ad esempio la condivisione di una certa risorsa hardware/software.

Il modello Client/Server prevede dunque due entità:

- **Client**: chi richiede il servizio.
- **Server**: chi offre il servizio.

Il modello Client/Server risolve il problema del rendez-vous (quando sincronizzare i processi comunicanti) definendo il Server come un processo sempre in attesa di richieste di servizio.

È di fatto un modello di comunicazione asimmetrica, molti ad 1:

- Il Cliente designa esplicitamente il destinatario.
- Il Servitore risponde al processo che ha effettuato una richiesta.

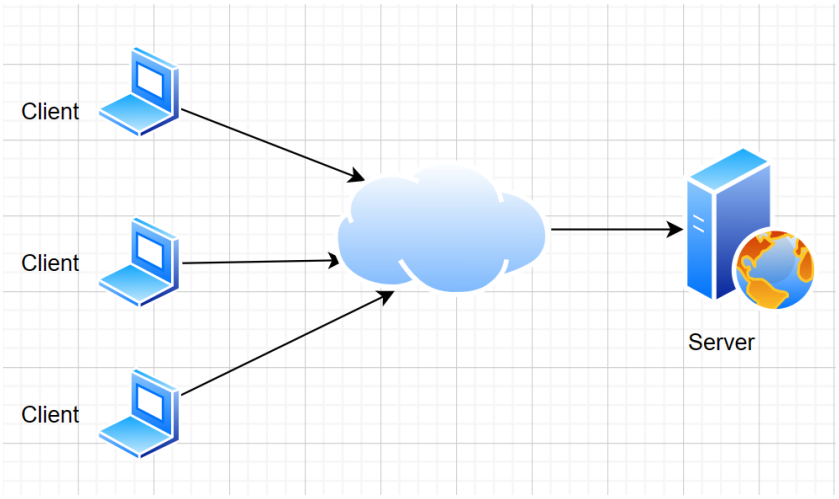


Figura 1 *- Modello Client Server*

Il Server deve dunque gestire richieste contemporanee da molti Client.

I tipi di interazione tra Client e Server sono fondamentalmente di due tipi:

- **connection oriented**: viene stabilito un canale di comunicazione virtuale prima di iniziare lo scambio dei dati (es. connessione telefonica).
- **connectionless**: non c'è connessione virtuale, ma semplice scambio di messaggi (es. il sistema postale).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La scelta tra i due tipi dipende dal tipo di applicazione e anche da altre caratteristiche proprie del livello di comunicazione sottostante (il livello di Trasporto); in Internet il livello di trasporto è TCP oppure UDP:

- **TCP**: con connessione, reliable (affidabile) e preserva l'ordine di invio dei messaggi.
- **UDP**: senza connessione, non reliable e non preserva ordine messaggi.

L'interazione tra Client e un Server può inoltre essere:

- **Stateful**: esiste lo stato dell'interazione, e quindi un messaggio dipende da quelli precedenti; un Server stateful ha migliore efficienza (dimensioni messaggi più contenute e migliore velocità di risposta del Server, presenta però problemi di replicazione).
- **Stateless**: non si tiene traccia dello stato, ogni messaggio è indipendente dagli altri; un Server stateless è più affidabile in presenza di malfunzionamenti (soprattutto causati dalla rete) ed è più semplice da progettare.

La scelta tra server stateless o stateful deve tenere in conto anche (e soprattutto) le caratteristiche dell'applicazione.

- Un'interazione stateless è possibile solo se il protocollo applicativo è progettato con operazioni idempotenti: operazioni idempotenti producono sempre lo stesso risultato, per esempio, un Server fornisce sempre la stessa risposta a un messaggio M indipendentemente dal numero di messaggi M ricevuti dal Server stesso.
- Quando si ha un'interazione stateful il Server deve poter identificare il Client.

È inoltre importante considerare il tema della "concorrenza" nel Client e nel Server.

I Client sono programmi sequenziali, eventuali invocazioni concorrenti supportate dal sistema operativo multitasking.

La concorrenza è invece cruciale per migliorare le prestazioni di un Server:

- un Server iterativo processa le richieste di servizio una alla volta; è dunque possibile un basso utilizzo delle risorse, in quanto non c'è sovrapposizione tra elaborazione ed I/O.
- Un Server concorrente gestisce molte richieste di servizio concorrentemente, cioè una richiesta può essere accettata anche prima del termine di quella (o quelle) attualmente in corso di servizio; le prestazioni ottenute sono migliori grazie alla sovrapposizione di elaborazione ed I/O. La gestione di processi concorrenti implica una analisi precisa della sincronizzazione nell'accesso alle risorse (principi di atomicità delle transazioni e di consistenza dei dati)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

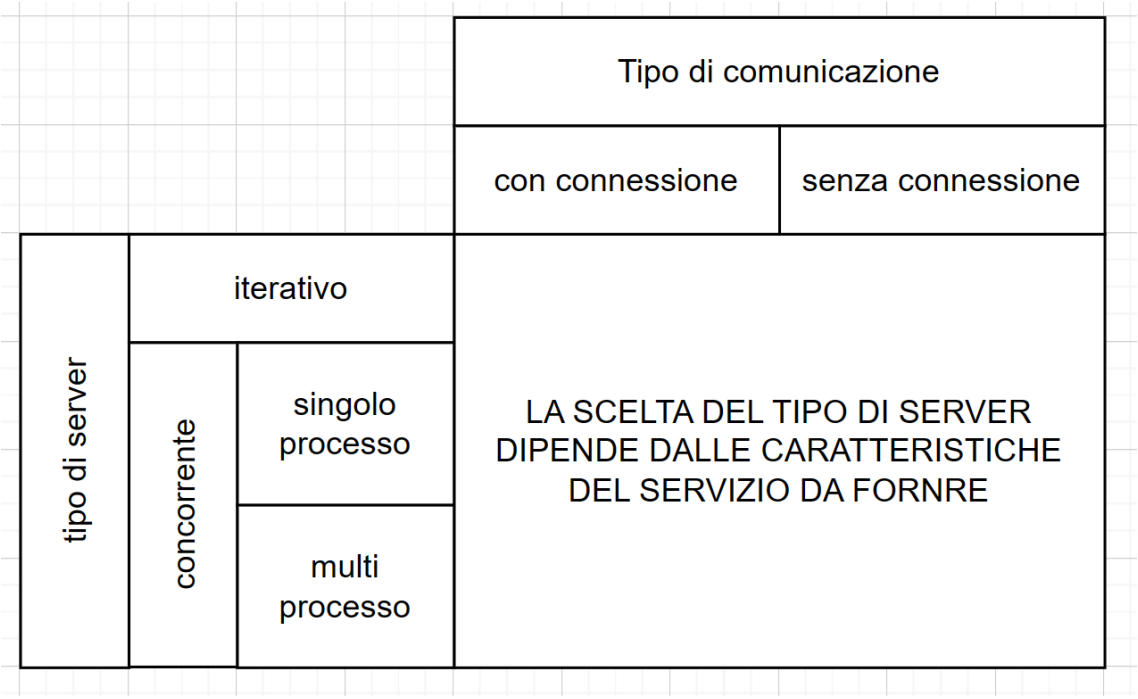


Figura 2 *- Tipologia di server*

Riportiamo di seguito alcuni esempi di Server in sistemi Client Server:

- **Web server**: per la gestione dell'interazione via web tra server e client.
- **Mail server**: per la gestione della posta elettronica.
- **File server**: per la condivisione dei file.
- **FTP server**: per la gestione dell'upload/download dei file.
- **Database server**: per la gestione di grandi moli di dati.
- **Groupware**: per la gestione d'informazioni riguardanti gruppi di lavoro.
- **Print server**: per la condivisione delle stampanti.
- […]

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. API (Application Program Interface)

Con API si intende un insieme di definizioni e protocolli per la creazione e l'integrazione del software applicativo.

Le API consentono ad un prodotto o servizio di comunicare con altri prodotti e servizi senza dover sapere come sono implementati: questo può semplificare lo sviluppo di app, risparmiando tempo e denaro. Quando si progettano nuovi strumenti e prodotti (o si gestiscono quelli esistenti), le API offrono: flessibilità; semplificano la progettazione, l'amministrazione e l'utilizzo; forniscono opportunità di innovazione.

Le API possono essere pensate come contratti, con documentazione che rappresenta un accordo tra le parti: se la parte 1 invia una richiesta remota strutturata in un modo particolare, ecco come risponderà il software della parte 2.

Immaginiamo di essere proprietari di una società di distribuzione di libri: potremmo offrire ai nostri clienti un'app cloud che consenta agli utenti di verificare la disponibilità del libro. Questa app potrebbe essere costosa da sviluppare, limitata dalla piattaforma e richiedere lunghi tempi di sviluppo e manutenzione continua. In alternativa, potremmo fornire un'API per verificare la disponibilità delle scorte.

Ci sono diversi vantaggi in questo approccio:

- Consentire ai clienti di accedere ai dati tramite un'API li aiuta ad aggregare le informazioni sul loro inventario in un unico posto
- Potremmo apportare modifiche ai nostri sistemi interni senza influire sui clienti, purché il comportamento dell'API non cambi
- Con un'API pubblicamente disponibile, gli sviluppatori che lavorano il sistema di distribuzione, i venditori di libri o terze parti potrebbero sviluppare un'app per aiutare i clienti a trovare i libri che stanno cercando. Ciò potrebbe comportare maggiori vendite o altre opportunità commerciali.

In sintesi, le API consentono di aprire l'accesso alle proprie risorse mantenendo sicurezza e controllo; la sicurezza delle API si basa su una buona gestione delle API, che include l'uso di un gateway API.

La connessione alle API e la creazione di applicazioni che consumano i dati o le funzionalità esposte dalle API possono essere eseguite con una piattaforma di integrazione distribuita che connette tutto, inclusi i sistemi legacy e l'Internet of Things (IoT).

Le API possono avere 3 tipi di release:

- **Private**: API per uso interno; ciò offre alle aziende il massimo controllo sulla propria API.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Partner**: API condivisa con partner commerciali specifici; ciò può fornire ulteriori flussi di entrate senza compromettere la qualità.
- **Public**: API disponibile per tutti; ciò consente a terze parti di sviluppare app che interagiscono con la propria API e possono essere una fonte di innovazione.

Quanto si parla di **Remote API**, ci stiamo riferendo ad API progettate per interagire attraverso una rete di comunicazione. Con "remote" si intende che le risorse manipolate dall'API si trovano da qualche parte al di fuori del computer che effettua la richiesta. Poiché la rete di comunicazione più utilizzata è Internet, la maggior parte delle API è progettata sulla base di standard web. Non tutte le API remote sono Web API, ma è giusto presumere che le Web API siano remote.

Le Web API in genere utilizzano HTTP per i messaggi di richiesta e forniscono una definizione della struttura dei messaggi di risposta. Questi messaggi di risposta in genere assumono la forma di un file **XML** o **JSON**. Sia XML che JSON sono formati preferiti perché presentano i dati in un modo che è facile da manipolare per altre app.

I due approcci architetturali che utilizzano maggiormente le remote API sono:

- **Service Oriented Archtecture** (**SOA**).
- **Microservice Architecture.**

SOA è il più antico dei 2 approcci, è nato come miglioramento delle app monolitiche: mentre una singola app monolitica fa tutto, alcune funzioni possono essere fornite da diverse app che sono liberamente accoppiate tramite un modello di integrazione.

Sebbene SOA sia, sotto molti aspetti, più semplice di un'architettura monolitica, comporta il rischio di cambiamenti a cascata in tutto l'ambiente se le interazioni tra i componenti non sono chiaramente comprese. Questa ulteriore complessità reintroduce alcuni dei problemi cui SOA ha cercato di porre rimedio.

Le architetture di microservizi sono simili ai modelli SOA nell'uso di servizi specializzati e debolmente accoppiati, ma si spingono ancora oltre nell'abbattere le architetture tradizionali: i servizi all'interno dell'architettura dei microservizi utilizzano un framework di messaggistica comune, (come le **RESTful API**) per comunicare tra loro senza complicate transazioni di conversione dei dati o ulteriori livelli di integrazione.

SOA è dunque un paradigma di progettazione che prevede la creazione di un insieme di servizi indipendenti che possono essere utilizzati da altri componenti dell'architettura e sono solitamente implementati come componenti software esposti attraverso un'interfaccia di programmazione (API) per consentire l'accesso a funzionalità specifiche.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I microservizi, invece, sono un'architettura software che si basa sull'idea di dividere un'applicazione in piccoli servizi autonomi che comunicano tra loro tramite un'interfaccia ben definita; sono solitamente più piccoli e meno complessi rispetto ai servizi SOA, e sono progettati per essere facilmente sostituiti e scalati.

In sintesi, mentre SOA è un paradigma generale per la creazione di servizi, i microservizi sono una specifica implementazione di SOA, che si concentra sulla creazione di servizi più piccoli e indipendenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. SOAP e REST

SOAP (**Simple Object Access Protocol**) è un protocollo utilizzato per la comunicazione tra sistemi in un'architettura **SOA** (Service-Oriented Architecture). Le API progettate con SOAP utilizzano XML per il formato dei messaggi e ricevono le richieste tramite HTTP o SMTP. SOAP semplifica la condivisione delle informazioni per le app in esecuzione in ambienti diversi o scritte in lingue diverse.

Le API Web che aderiscono ai vincoli dell'architettura **REpresentational State Transfer** (**REST**) sono denominate **RESTful API** e sono prevalentemente usate per creare **microservices**. REST differisce da SOAP in modo fondamentale: SOAP è un protocollo, mentre REST è uno stile architetturale: ciò significa che non esiste uno standard ufficiale per le web RESTful API, tuttavia tali API si basano sull'utilizzo di standard **HTTP** (Hypertext Transfer Protocol) come protocollo di comunicazione tra client e server, ed utilizza le principali verbi HTTP (GET, POST, PUT, DELETE) per effettuare operazioni **CRUD** (Create, Read, Update, Delete) sulle risorse identificate da **URI** (Uniform Resource Identifier). Inoltre, REST prevede l'uso di formati standard come **JSON** o **XML** per la rappresentazione delle risorse.

Come definito nella dissertazione di Roy Fielding "Architectural Styles and the Design of Network-based Software Architectures", le API sono RESTful fintanto che rispettano i 6 vincoli guida di un sistema RESTful:

1. **Architettura Client-Server**: l'architettura REST è composta da client, server e risorse e gestisce le richieste tramite http
2. **Stateless**: nessun contenuto del client viene memorizzato sul server tra le richieste. Le informazioni sullo stato della sessione sono invece conservate presso il client
3. **Cacheability**: la memorizzazione nella cache può eliminare la necessità di alcune interazioni Client-Server
4. **Layered System**: le interazioni Client-Server possono essere mediate da livelli aggiuntivi. Questi livelli potrebbero offrire funzionalità aggiuntive come bilanciamento del carico, cache condivise o sicurezza.
5. **Code on demand** (optional): i server possono estendere la funzionalità di un client trasferendo codice eseguibile.
6. **Uniform Interface**: questo vincolo è fondamentale per la progettazione delle API RESTful e include 4 aspetti:
    a. **Identificazione delle risorse nelle richieste**: le risorse sono identificate nelle richieste e sono separate dalle rappresentazioni restituite al cliente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

b. **Manipolazione delle risorse tramite rappresentazioni**: i client ricevono file che rappresentano risorse. Queste rappresentazioni devono avere informazioni sufficienti per consentire la modifica o la cancellazione.
c. **Messaggi autodescrittivi**: ogni messaggio restituito a un client contiene informazioni sufficienti per descrivere come il client dovrebbe elaborare le informazioni.
d. **Hypermedia come motore dello stato dell'applicazione**: dopo l'accesso a una risorsa, il client REST dovrebbe essere in grado di scoprire tramite collegamenti ipertestuali tutte le altre azioni attualmente disponibili.

Questi vincoli possono sembrare molti ma sono molto più semplici di un protocollo prescritto. Per questo motivo le API RESTful stanno diventando sempre più diffuse rispetto a SOAP.

Negli ultimi anni, la specifica OpenAPI è emersa come standard comune per la definizione delle API REST. OpenAPI stabilisce un modo indipendente dal linguaggio per gli sviluppatori di creare interfacce API REST in modo che gli utenti possano comprenderle con il minimo sforzo di congettura.

Un altro standard API emergente è GraphQL, un linguaggio di query e un runtime lato server che è un'alternativa a REST. GraphQL dà la priorità a fornire ai clienti esattamente i dati richiesti e non di più. In alternativa a REST, GraphQL consente agli sviluppatori di creare richieste che estraggono dati da più origini dati in una singola chiamata API.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. La popolarità di Rest

Ma come si è arrivati alla popolarità di REST?

Tale popolarità nasce dalla popolarità (nel tempo) delle web API rispetto ad altri modelli come il **Message-oriented middleware (MOM)**.

L'esigenza iniziale era infatti gestire una chiara necessità degli utenti in ambito web: molto spesso, l'utente non ha necessità di una risposta a seguito di una richiesta, specie se l'elaborazione implica molto tempo. Sarebbe il caso, allora, di valutare un modello di comunicazione asincrono, dove l'utente effettua una richiesta ma non attende la risposta. La risposta sarà elaborata, e successivamente inoltrata al richiedente. Da questa considerazione prendeva origine una forma di middleware conosciuta come **Message-oriented middleware (MOM)**: infrastruttura software o hardware che gestisce l'interoperabilità attraverso lo scambio di messaggi; permettono un accoppiamento lasco tra i componenti perchè l'invio dei messaggi è asincrono e non implementano sistemi per la gestione di uno stato condiviso.

Da questo approccio nascono due tipi di modelli di comunicazione asincrona:

- **Publish/Subscribe** (**Pub/Sub**): meccanismo di distribuzione molti a molti in cui messaggi sono instradati in un certo canale in cui n altri client possono restare in ascolto.
- **Point-to-Point**: i messaggi prodotti da un'entità vengono instradati verso uno specifico destinatario, solitamente in una coda FIFO.

Il primo caso è il caso broadcast, il producer non conosce i consumer. Nel secondo invece, il producer deve conoscere il consumer (ed il suo indirizzo).

Tale modello ha ben presto lasciato il posto a modelli basati su web API che hanno portato alla definizione di due approcci architetturali:

- **Service Oriented Archtecture** (**SOA**).
- **Microservice Architecture.**

L'approccio a microservices mediante utilizzo di RESTful API ha ben presto prevalso per i vantaggi indicati in precedenza e che potremmo riassumere nei seguenti punti:

- **Scalabilità**: il server non deve tenere in memoria alcuna informazione riguardo i client esistenti (e i servizi possono essere replicati su più server).
- **Reliability**: in caso di disastro non c'è necessità di recuperare lo stato condiviso ma soltanto l'applicazione stessa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Più **facile** implementazione.
- **Visibilità**: ciascuna richiesta è atomica e può essere monitorata facilmente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- F. Bergamasco - WEB APPLICATIONS AND TECHNOLOGIES - CT0142.
- What is an API? (redhat.com)
- https://www.evemilano.com/protocolli-http/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).